

Dogfight-aware BT Generator Platform — 설계 계획서 v6.0

최초 작성: 2026-04-05

최종 갱신: 2026-04-13 (v6.0: 닫힌 루프 학습 파이프라인으로 재정의)

목표: 1:1 공중전 교전 결과로부터 적의 의도에 따른 최적 대응을 자동 도출하는 BT generator platform 연구

비유: Fuzzing framework의 1:1 dogfight-aware 버전

목차

1. 프로젝트 정의 (Research Framing)
2. 문제 정의 & 성공 기준
3. 이론적 배경
4. 파이프라인 아키텍처 — 3-Stage × 4-Layer
5. Stage ① EXPLORE — 탐색
6. Stage ② LEARN — 결정기 개발
7. Stage ③ APPLY — 실전 & 피드백
8. Hypothesis Mining 아키텍처
9. CMA-ES의 재정의된 역할
10. 지식 DB 구조
11. 현재 상태 & 다음 Sprint
12. 부록

1. 프로젝트 정의

1.1 연구 프레임밍

본 프로젝트는 단일 BT를 만드는 것이 아니라 BT generator platform을 만드는 연구이다.

- 대상 도메인: 1:1 공중전 (dogfight), JSBSim 기반 시뮬레이션
- 유사 프레임워크: Fuzzing framework (입력 공간 탐색 → 버그 발견 → 수정 → 순환)
- 차이점: 입력이 "random byte stream"이 아니라 "적 기동 패턴"이고, 버그가 "crash"가 아니라 "패배/교착"이다

1.2 플랫폼의 역할 (파이프라인 한 문장)

과거 교전 데이터로 가설을 세우고, 대규모 시뮬로 검증해 각 기동의 최적값과 적 의도 모델을 얻고, 실전에서 의도를 추론하여 카운터 기동을 선택한다. 결과는 다시 데이터셋으로 환류된다.

1.3 왜 정적 최적화가 아닌가

이전 v5.0 계획은 "CMA-ES로 single best BT를 찾는다" 였으나, 이 세션에서 다음 증거가 누적되며 파이프라인 방향을 전환:

- 관찰 1: CMA-ES cycle_2 best가 hand-designed v5.1보다 실제로 약함 (특정 상대 조합에서 0% WR)
- 관찰 2: 동일 BT가 어떤 상대는 완승, 어떤 상대는 draw — **Pareto trade-off 존재**
- 관찰 3: Static BT는 runtime에 적의 행동 변화에 적응 불가 (BFM은 본질적으로 non-stationary)
- Jensen 부등식: $\mathbb{E}_o[\max_x f(x, o)] \geq \max_x \mathbb{E}_o[f(x, o)]$
→ adaptive BT는 theoretical upper bound가 static보다 항상 $\geq$

따라서 새 파이프라인은:

1. Static 최적화 → **Hypothesis-driven 진화 + Runtime adaptation**
2. Black-box CMA-ES → 해석 가능한 1D/2D 스윙 + 가설 검증
3. Single best BT → **Intent-aware counter selector + decider**

2. 문제 정의

2.1 공식 목표

주어진 것:

- 교전 데이터셋 $\mathcal{D} = \{(o_i, a_i, r_i)\}$, 관측·액션·결과 튜플
- 상대 분포 $\mathcal{O}$ (695 BT로 근사)
- BT 파라미터 공간 $\mathcal{X}$

목표 (세 개의 결합):

1. **노드 최적값 발견**: 각 BFM 노드 n 에 대해 목적에 부합하도록 동작하는 파라미터 θ_n^* 를 데이터로부터 학습.

$$\theta_n^* = \arg \max_{\theta} \mathbb{P}[\text{node achieves BFM purpose} \mid \theta, \text{obs}]$$

2. **Intent 분류기 학습**: 관측 시퀀스 $o_{1:K}$ 로부터 적의 의도 $i \in \mathcal{I}$ 를 예측.

$$\hat{\phi} = \arg \max_{\phi} \mathbb{P}[i \mid o_{1:K}, \phi], \quad \mathcal{I} = \{\text{GUN_ATTACK, PURSUIT, DEFENSIVE, ENERGY, NEUTRAL_CIRCLE, NEUTRAL_SCISS}\}$$

3. **Counter Selector (결정기)**: intent i 에 대한 최적 대응 노드 $c(i)$.

$$c^*(i) = \arg \max_{n \in \mathcal{N}} \mathbb{P}[\text{win} \mid \text{enemy intent} = i, \text{own action} = n]$$

2.2 왜 어려운가

어려움	설명	대응
Non-stationary	적이 우리 전략에 적응	Feedback loop (runtime 환류)
Noisy evaluation	같은 BT여도 매치마다 결과 다름 (~25% FP non-determinism)	Wilson CI + 대규모 매치
Partial observability	상대 의도는 직접 관측 불가 → 추론 필요	Intent classifier (ProtoNet)
Compositional	"좋은 노드" × "좋은 selector" 조합	각 구성요소 독립 검증
Interpretability	블랙박스 solution은 디버깅 불가	Hypothesis-driven (해석 가능)

2.3 성공 기준

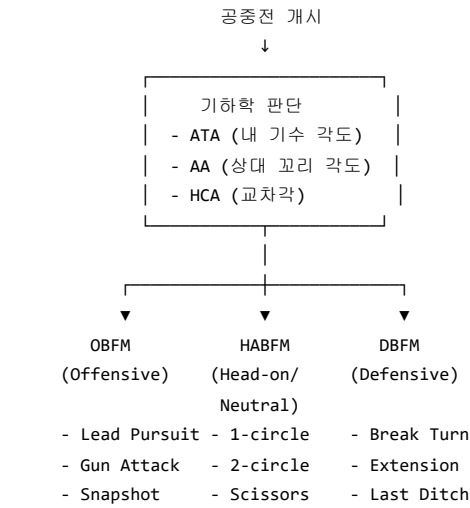
지표	목표	측정 방법
Universal Win Rate	≥ 75% (695 풀, 10R)	Wilson CI ±1.18%
Worst-case layer WR	≥ 55%	per-layer 분석
Intent classifier accuracy	≥ 75%	per-class accuracy
Counter_table coverage	6/6 intent	each intent has verified counter
Draw 비율	≤ 15%	교착 최소화
test_suite	5/5	자동 검증
가설 검증 사이클	≥ 10 개 CONFIRMED	hypothesis_tracker

3. 이론적 배경

3.1 BFM (Basic Fighter Maneuvers) — Shaw의 분류

출처: Robert L. Shaw, *Fighter Combat: Tactics and Maneuvering* (1985)

공중전은 세 가지 기하학적 단계로 분류된다:



각 기동은 **고유한 목적**을 가지며, 이 목적이 달성되려면 **관측값 변화에 따라 물리량(반경, 속도, 고도)이 동적으로 조절**되어야 한다.

관측 단위 규약 (불변)

내부 관측은 **0~1로 정규화**되어 있음. 커스텀 노드에서 반드시 변환:

키	내부 범위	실제 단위	변환식
ata_deg , aa_deg , hca_deg	0~1	0~180°	val × 180
tau_deg , relative_bearing_deg	-1~1	-180~180°	val × 180
distance_ft , closure_rate_kts , ego_altitude_ft	raw	ft / kts	변환 불필요

⚠ 이 규약 위반 = **BUG-4 수준의 silent failure**. 조건이 항상 False/True로 평가됨.

WEZ (Weapon Engagement Zone)

- Distance: 152 ~ 914 ft
- ATA: < 12°
- Base DPS: 25
- Hard Deck: 1,000 ft 이하 즉시 패배

3.2 목적 기반 BFM 구현 (Purpose-driven Nodes)

각 BFM 노드는 다음 형식을 따른다:

BFM Node = (Purpose, Invariants, Feedback Laws)

Purpose : 이 기동이 달성하려는 전술적 목표 (한 문장)
Invariants : 동작이 올바른지 검증할 수 있는 조건 (예: "ATA는 감소해야 함")
Feedback Laws : 관측 변화에 따라 물리량을 어떻게 조절하는지 규칙

예시 — SmarthHighYoYo:

- Purpose:** 적 turn circle 안쪽에서 초과 closure를 수직 에너지로 변환
- Invariant:** climb 중 거리가 벌어지면 yoyo 반경이 과대 → 즉시 dive
- Feedback Laws:**
 - e_diff > 3000ft → 강제 DIVE (폭주 방지)
 - dist 증가 중 → 즉시 DIVE (반경 과대 감지)
 - |closure| < 100 kts → DIVE (정상화)
 - 그 외 → CLIMB 유지

3.3 Wilson Score Interval — 통계적 검증 기준

$$CI_{95\%} = \frac{\hat{p} + \frac{z^2}{2n} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \quad z = 1.96$$

정규근사 대비 장점: $\hat{p} = 0$ 또는 1일 때도 올바른 CI (coverage guarantee).

실용 해석:

n 매치	CI margin @ $p = 0.5$	의미
10	±31%	의미 없음
100	±10%	단일 가설 검증 최소 기준
695 (×1R)	±3.7%	layer별 진단 가능
6,950 (×10R)	±1.18%	Universal claim 가능
34,750 (×50R)	±0.53%	per-opponent 진단 가능

Validation Threshold (가설 검증 게이트):

- 가설이 CONFIRMED되려면:
- 1. 최소 **100 매치** 이상으로 테스트
 - 2. Wilson CI lower bound > baseline_wr
 - 3. 기록된 hypotheses.json1 에 태그

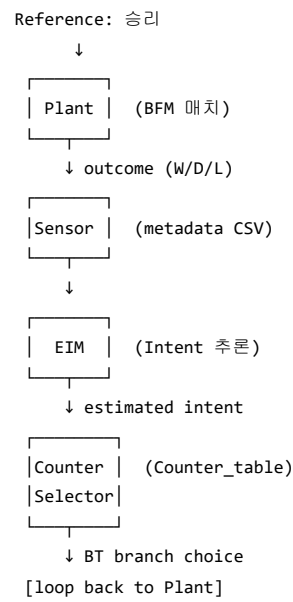
3.4 LHS (Latin Hypercube Sampling) — 연속 파라미터 탐색

5-dim 연속 파라미터를 Grid Search로 스윕하면 $5^5 = 3125$ 개가 필요.

LHS는 n=80개로 각 1차원 marginal을 균등 커버.

3.5 Closed-loop Feedback Control (신규 추가)

본 파이프라인은 기본적으로 닫힌 루프 제어 시스템이다:



Feedback 루프가 없으면 "열린 루프 open-loop" → 어떤 파라미터 튜닝도 non-stationary 환경에서 점근적 최적에 도달하지 못한다.

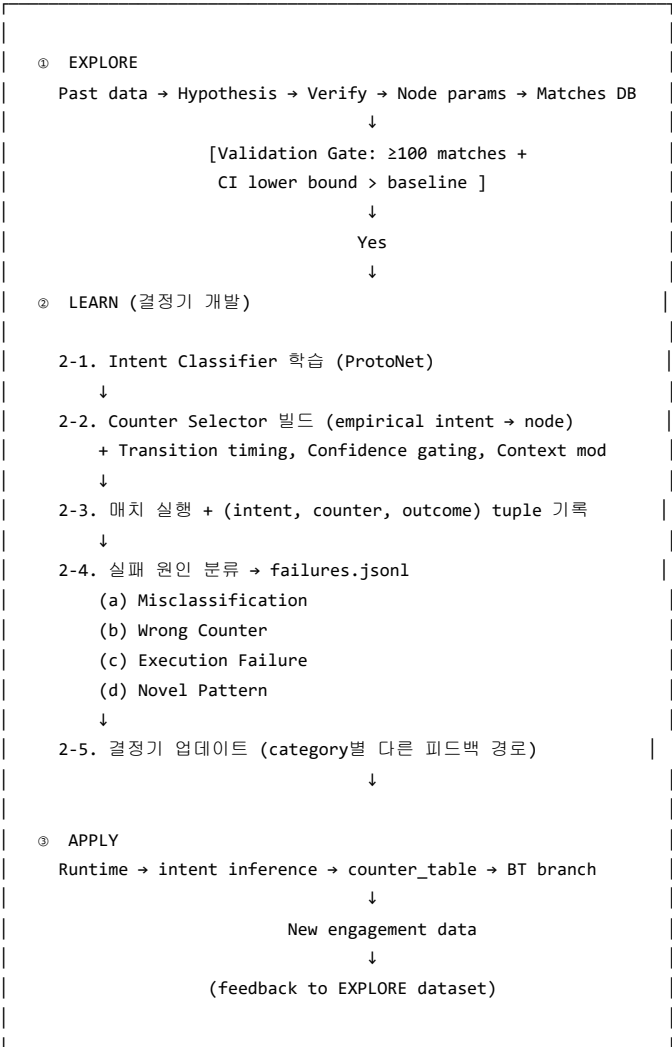
4. 파이프라인 아키텍처

4.1 2축 구조: Pipeline Stage × Implementation Layer

		IMPLEMENTATION LAYER			
PIPELINE STAGE		Measure	Correct	Search	Pool
		evaluate.py	test_suite	(sweep/ES)	opp_pool
		Wilson CI	BUG fix	node tune	695 BT
④EXPLORE (data → hyp)		metadata	runner	1D sweep	training
		CSV	drift fix	param	pool:
		analyze	obs unit	tuning	L1-L5
		compare			
④LEARN (decider)		Wilson gate	intent	counter_	holdout:
		hypothesis	label	table	L6
		tracker	verification	refinement	
④APPLY (real-time + feedback)		runtime	EIM runtime	BT branch	full pool
		telemetry	integration	selection	validation
		+ replay	safe fallback	via intent	695 × 10R

해석: v5.0 Phase 1-4(세로축, Layer)는 유지. 그 위에 Pipeline Stage (가로축) 을 추가하여 "시간적 흐름 × 구현 책임" 을 분리.

4.2 Feedback Loop



5. Stage ① EXPLORE

목표: 과거 교전 데이터로부터 가설을 세우고 대규모로 검증하여 각 BFM 노드의 최적값과 작동 조건을 찾는다.

5.1 Sub-steps

단계	도구	산출물
1-1. 데이터 수집	collect_phase1.py	logs/metadata/<ts>_<a1>_vs_<a2>_meta.csv (per-tick 30 columns)
1-2. 분석	analyze_metadata.py , analyze_acmi.py	매치별 metric 집계
1-3. 가설 생성	hypothesis_miner.py (§8)	후보 가설 queue
1-4. 대규모 검증	evaluate.py + 695 풀	≥100 매치 per hypothesis
1-5. 노드 최적값 도출	1D sweep (via hypothesis_tracker)	각 TUNABLE_PARAMS 최적치
1-6. Verdict 기록	hypothesis_tracker.py	hypotheses.jsonl 추가

5.2 Data Collection — Metadata CSV가 Primary

collect_phase1.py 가 생성하는 per-tick CSV가 **파이프라인의 raw data source**. ACMI는 시각화 보조 (Tacview에서 사람이 보기 위한 용도).

CSV 컬럼 (30개):

```
step, agent_id, tree_name, bfm_situation,
distance_ft, ata_deg, aa_deg, hca_deg, relative_bearing_deg,
ego_altitude_ft, ego_vc_kts, specific_energy_ft, ps_fts, energy_diff_ft,
closure_rate_kts, turn_rate_degs, alt_gap_ft,
in_wez, enm_in_wez, in_39_line, overshoot_risk,
energy_advantage, alt_advantage, spd_advantage,
tc_type, side_flag,
ego_health, enm_health, ego_damage_dealt, enm_damage_dealt
```

왜 CSV인가:

- 매 tick 관측값 전체 (30 column) → 모든 패턴 탐지 가능
- BFM 전이 분석 (UNKNOWN → OBFM 등) 가능
- active_node 추적 → 어느 노드가 실제로 발동하는지
- analyze_metadata.py 의 UNKNOWN 서브분류 재사용 가능

5.3 노드 최적값 도출 — 1D 스윙 > CMA-ES

각 BFM 노드의 TUNABLE_PARAMS 파라미터는 **순차 1D 스윙** 으로 탐색:

```
# 예시: SmartHighYoYo의 energy_force_dive_ft 탐색
baseline_wr = evaluate(current_bt, 100 matches)

for val in [2000, 2500, 3000, 3500, 4000]:
    modified_bt = set_param("SmartHighYoYo", "energy_force_dive_ft", val)
    wr = evaluate(modified_bt, 100 matches)
    register_hypothesis(
        f"SmartHighYoYo.energy_force_dive_ft={val}",
        result_wr=wr,
        baseline_wr=baseline_wr,
    )

best_val = argmax(wrs)
```

1D 스윙의 장점:

- 해석 가능: "이 값이 왜 좋은지" 수치로 설명
- Hypothesis DB와 호환: 각 스윙이 독립 가설로 기록
- 낮은 매치 비용: 5 values × 100 matches = 500 매치 per param vs CMA-ES 400 매치 for 104-dim
- 상호작용 누락 리스크: 파라미터 간 상관관계 큰 경우 → Stage ② 이후 CMA-ES로 final polish

5.4 Feedback Gate

EXPLORE 출력: 각 노드의 최적 파라미터 + Verdict DB

다음 단계로 진행 조건:

- 최소 10개 가설이 CONFIRMED 상태
- 주요 BFM 노드 (Lead, Pure, Lag, Break, HighYoYo, GunAttack) 파라미터 결정됨
- Metadata CSV ≥ 5,000 매치 누적 (Intent 학습용 샘플 확보)

충족 시 → LEARN 진입

불충족 시 → 더 수집, 더 가설 생성

6. Stage ② LEARN

목표: 검증된 데이터로부터 "분류기 + 결정기" 를 만든다. 결정기는 의도 → 최적 대응 → 승리 검증 → 실패 원인 분류 → 재학습 의 5-step 루프로 정의.

6.1 2-1. Intent Classifier 학습

항목	값
입력	K-step 관측 window (K=20 tick = 4초)
출력	intent class 확률 분포
모델	ProtoNet (GRU encoder + prototype distance)
학습 도구	train_eim.py
데이터	EXPLORE 누적 metadata CSV
검증 기준	per-class accuracy ≥ 75%, prototype 분리도

Failure 시 피드백: 특정 class 정확도 낮음 → EXPLORE로 복귀하여 해당 intent 데이터 보강.

6.2 2-2. Counter Selector (결정기)

데이터 구조: logs/knowledge/counter_table.json

```
{
  "GUN_ATTACK": {
    "best": "SmartBreakTurn",
    "wr": 0.85,
    "n": 120,
    "ci_lower": 0.77,
    "min_hold_ticks": 15,
    "exit_condition": "closure_dropped_below_100",
    "conf_high_variant": "SmartBreakTurn aggressive params",
    "conf_low_variant": "ExtensionBreak safe",
    "context_overrides": {
      "close_range": "LastDitch"
    }
  },
  "PURSUIT": { ... },
  ...
}
```

구축 방법:

- 1. EXPLORE 누적 매치에서 매 tick (intent_predicted, active_node, eventual_outcome) 기록
- 2. 각 (intent, node) 쌍별 승률 집계
- 3. Wilson CI lower bound 기준으로 intent별 best node 선택
- 4. Transition timing / confidence / context 파라미터는 1D 스윙으로 추가 학습

3가지 확장 (모두 1D 스윙으로 empirical):

확장	질문	탐색
Transition timing	언제 counter 교체?	N consecutive ticks ∈ {5, 10, 15, 20, 30}
Confidence gating	신뢰도 낮을 때 어떻게?	2 bands: high/low, threshold ∈ {0.5, 0.6, 0.7, 0.8}
Context modulation	같은 intent, 다른 상황?	(intent, dist_bin, energy_sign) → counter

시작은 가장 단순하게 (고정 N=15, 단일 threshold=0.7, intent only). 실패 분석(2-4)이 더 세밀한 분기 필요성을 드러내면 확장.

6.3 2-3. 실행 & 관찰

매 tick 기록:


```
{
  "match_id": "...",
  "tick": 432,
  "intent_predicted": "PURSUIT",
  "intent_confidence": 0.78,
  "counter_selected": "SmartLagPursuit",
  "observation_summary": {
    "ata": 45.2,
    "distance": 2300,
    "energy_diff": +800,
    ...
  },
  "eventual_outcome": "WIN"
}
```

6.4 2-4. 원인 파악 (Cause Analysis) — 핵심 피드백 포인트

승리가 아닐 때, 4가지 카테고리로 자동 분류:

원인	판정 조건	되먹임 대상
(a) Misclassification	intent_predicted ≠ ground_truth_intent (post-hoc 활성노드 기반)	2-1 재학습 — 더 많은 데이터
(b) Wrong Counter	intent 맞췄으나 counter 승률이 낮음	2-2 counter_table 수정
(c) Execution Failure	counter 맞췄으나 실제 노드 파라미터가 부적절	EXPLORE 1-5 — 노드 재튜닝
(d) Novel Pattern	관측이 어느 intent class에도 맞지 않음	EXPLORE 1-1 — 새 class 추가

DB 적재: logs/knowledge/failures.jsonl

```
{
  "ts": "2026-04-13T...",
  "match_id": "...",
  "outcome": "LOSS",
  "cause_category": "(b) Wrong Counter",
  "details": {
    "intent_predicted": "PURSUIT",
    "counter_used": "SmartLeadPursuit",
    "better_counter_candidate": "SmartLagPursuit",
    "evidence": "lag was chosen in 30 similar situations with 78% WR"
  },
  "feedback_target": "counter_table"
}
```

6.5 2-5. 결정기 업데이트

트리거: failures.jsonl 에 특정 cause category가 N건 이상 누적 → 자동 업데이트.

Category	업데이트 방법
(a) Misclassification	train_eim.py --finetune with new labeled data
(b) Wrong Counter	counter_table의 해당 intent best를 교체 후 A/B 테스트
(c) Execution Failure	해당 노드 TUNABLE_PARAMS 재스왑
(d) Novel Pattern	INTENT_CLASSES 확장 → EXPLORE 재시작

7. Stage ③ APPLY

목표: 실전 매치에서 결정기를 가동하고, 결과를 자동으로 데이터셋에 환류한다.

7.1 Runtime Flow

매 BT tick (0.2초):

1. 관측 수집 (29 features)
2. Intent inference:
 - OnlineIntentTracker가 sliding window (K=20)에 obs 추가
 - 매 update_interval tick마다 ProtoNet predict
 - confidence < threshold → UNKNOWN (fallback)
3. Counter lookup:
 - counter_table[intent] → node name
 - min_hold_ticks 체크 (oscillation 방지)
 - confidence band에 따라 variant 선택
4. BT branch 선택 (EnemyIntentIs[X] → counter[X])
5. Action 실행 → [alt, hdg, vel] 3-tuple → JSBSim
6. 매치 종료 시 metadata CSV 저장 → EXPLORE dataset 자동 환류

7.2 BT 구조 (Intent-based Branching)

옵션 A (정적, 권장) — counter_table에서 YAML 자동 생성:

```
tree:
  type: Selector
  children:
    - type: Sequence
      name: HardDeckAvoidance
      children:
        - {type: Condition, name: BelowHardDeck, params: {threshold_ft: 1200}}
        - {type: Action, name: ClimbTo, params: {target_altitude_ft: 3000}}

    - type: Sequence
      name: GunEngagement
      children:
        - {type: Condition, name: IsWEZOpportunity}
        - {type: Action, name: SmartGunAttack}

    # Intent-driven branches (counter_table로부터 자동 생성)
    - type: Sequence
      name: CounterGunAttack
      children:
        - {type: Condition, name: EnemyIntentIs, params: {intent: "GUN_ATTACK", min_confidence: 0.7}}
        - {type: Action, name: SmartBreakTurn, params: {...}}

    - type: Sequence
      name: CounterPursuit
      children:
        - {type: Condition, name: EnemyIntentIs, params: {intent: "PURSUIT", min_confidence: 0.7}}
        - {type: Action, name: SmartLagPursuit, params: {...}}

    # ... (intent class 수만큼 반복)

    # Fallback
    - type: Action
      name: SmartLeadPursuit
```

자동 생성기: tools/build_bt_from_counter_table.py (TBD)

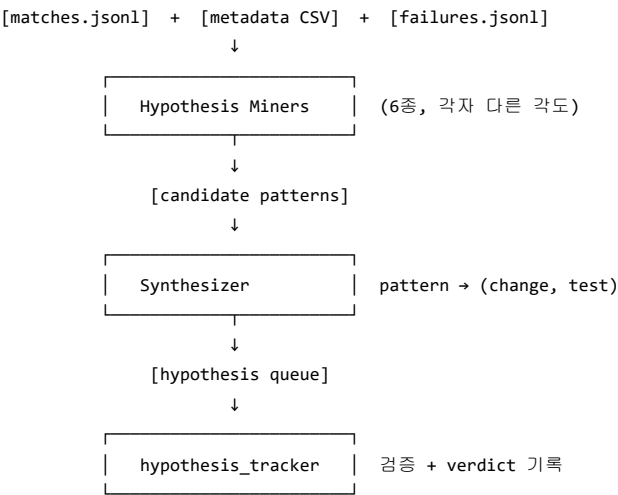
7.3 Feedback to EXPLORE

매 매치 종료 시 `collect_phase1.py` 가 생성한 `metadata CSV`가 `logs/metadata/` 에 저장되며, 이는 EXPLORE 단계의 데이터 소스와 동일. **수동 개입 없이 자연 환류.**

8. Hypothesis Mining

문제: CMA-ES가 자동화했던 "어디를 고칠지" 를 hypothesis-driven에서는 사람이 찾아야 함. 따라서 **Hypothesis Miner** 층이 파이프라인의 진짜 병목이 되며, 이를 자동화해야 한다.

8.1 Architecture



8.2 6가지 Miner

Miner 1: Rigid-behavior Detector

"행동이 관측 변화에 반응하지 않는 구간"

- 입력: `metadata CSV per-tick`
- 출력: `(node, observation_drift, tick_range)`
- 이미 구현: `tools/find_rigid_behavior.py`

Miner 2: Outcome-Discriminating Features

"승/패를 가장 크게 가르는 feature"

```
for metric in metrics:
    effect_size = (mean_win - mean_loss) / std_all
    if abs(effect_size) > 0.5:
        yield Hypothesis(target=metric, priority=abs(effect_size))
```

Miner 3: Threshold Discovery

"어떤 metric의 임계값에서 WR이 급변하나"

```
for t in candidate_thresholds:
    wr_below = win_rate(filter(matches, metric < t))
    wr_above = win_rate(filter(matches, metric >= t))
    if abs(wr_below - wr_above) > 0.3:
        yield Hypothesis(
            f"조건 노드 IsMetricExceeded({metric}, {t}) 추가",
            priority=abs(wr_below - wr_above)
        )
```

중요: 이 miner가 H1을 자동으로 발견했을 가설. IsLostPursuit 에 dist > 2000 조건이 이 miner의 출력.

Miner 4: Failure Mode Clusterer

"패배가 공유하는 공통 패턴"

- LOSS 매치를 feature vector로 → k-means/DBSCAN
- 각 cluster = 독립적 실패 모드
- 제안: 해당 cluster의 feature 조합을 감지하는 조건 + 회피

Miner 5: Node Usage Imbalance

"어느 노드가 과소/과대 사용되나"

- fire_pct < 1% → 조건 relax 또는 순서 변경
- fire_pct > 60% + low WR → 조건 좁힘, 다른 node 필요
- alpha2 화귀의 HeadOnBreak 44% 패턴을 자동 탐지했을 miner

Miner 6: Counter-factual

"이 loss와 가장 비슷한 win의 차이는?"

- LOSS match → feature space에서 nearest WIN 찾기
- diff feature = 결정적 차이
- 제안: diff feature를 유도하는 조건/액션

8.3 Synthesizer

각 miner output을 "코드 변경 제안"으로 번역:

```
SYNTHESIS_TEMPLATES = {
    "threshold_discovery": lambda metric, t:
        f"Add IsMetricExceeded({metric}, threshold={t}) condition + branch",
    "node_underused": lambda node:
        f"Relax condition threshold for {node}",
    "rigid_behavior": lambda node, obs:
        f"Add feedback loop to {node} based on {obs}",
    "failure_cluster": lambda feat_combo:
        f"Detect {feat_combo} + counter action",
    ...
}
```

우선순위 랭킹: expected_impact × (1 / test_cost) × novelty_score

8.4 자동화 경계

자동	반자동	수동
패턴 탐지	코드 변경 제안	실제 코드 수정
가설 검증 (매치 실행)	LLM/사람 리뷰 후 적용	BT 구조 재설계
verdict 기록		새 BFM 이론 추가

9. CMA-ES의 재정의된 역할

9.1 결정 요약

CMA-ES는 더 이상 파이프라인의 central 도구가 아니다. "Final polish tool" 로 강등.

9.2 왜 중심에서 빠졌는가

원인	증거
Hand-designed > CMA-ES 관찰	cycle_2 CMA-ES best (104-dim, 400 evals)가 v5.1 hand-designed보다 실측 성능 낮음
Hypothesis-driven과 궁합 불일치	CMA-ES 결과는 "왜 이 값인지" 해석 불가 → 가설 검증 방식과 충돌
Purpose-driven 노드가 제공하는 의미	각 파라미터가 공학적 의미를 가짐 → 블랙박스 탐색 필요 없음
Noisy fitness	40-sample × 1R fitness가 noise 큼 → local optimum에 갇힘

9.3 언제 CMA-ES를 쓸 것인가

유일한 용도: 파이프라인 마지막 단계에서, 이미 모든 구조/하이퍼파라미터가 결정된 상태에서 **상관된 연속 파라미터 묶음을 jointly 튜닝**할 때.

예:

- SmartLeadPursuit 의 dist_widen_thresh + ata_worsen_thresh + overshoot_closure + sprint_ata_max_deg 4개 → 서로 상호작용 있음
- 1D 스윙으로는 최적 조합 놓칠 수 있음
- Hypothesis가 이미 모두 CONFIRMED된 상태에서 "**최종 0.5~2%p 추가 WR 짜내기**" 용도

9.4 기존 인프라 처분

파일	운영
tools/adaptive_optimizer.py	유지 — final polish 호출용
tools/generate_opponent_pool.py	유지 — EXPLORE/APPLY 모두에 필수
logs/cycle_N/ 구조	유지 — 가설 사이클 tracking
TUNABLE_PARAMS auto-discovery	유지 — 1D 스윙도 이 매커니즘 재사용

즉 삭제할 코드는 없다. 역할만 재정의.

10. 지식 DB 구조

10.1 파일 레이아웃

```
ai-combat-sdk/
├── logs/
│   ├── metadata/                                # ← EXPLORE raw data (primary source)
│   │   ├── <ts>_<a1>_vs_<a2>_meta.csv  # per-tick 30 cols
│   │   └── <ts>_<a1>_vs_<a2>_meta_result.json
│   ├── knowledge/                               # ← 집계 + 가설 + 패턴 + 결정기
│   │   ├── matches.jsonl                # 매치별 집계
│   │   ├── hypotheses.jsonl            # 가설 + verdict
│   │   ├── situations.jsonl            # 유리/불리/교착 패턴
│   │   ├── counter_table.json          # ★ 결정기 본체 (LEARN 산출물)
│   │   ├── failures.jsonl              # ★ 실패 원인 DB (LEARN 피드백 트리거)
│   │   └── class_coverage.json          # intent class별 샘플·정확도
│   ├── cycle_N/                               # 사이클별 실험 기록
│   │   ├── best.yaml
│   │   ├── validation.json
│   │   ├── diagnosis.md
│   │   └── changeset.md
│   └── replays/                               # ACMI (사람이 Tacview로 볼 때만)
├── models/
│   └── intent_model.pt                        # ProtoNet (LEARN 산출물)
└── submissions/
    └── GwangPung/                            # 대회 제출 (self-contained)
```

10.2 데이터 태깅 규약

matches.jsonl 의 각 레코드는 다음 태그를 가져야 한다:

```
{
  "timestamp": "...",
  "source": "metadata_csv" | "acmi",
  "hypothesis_id": "H1" | null,
  "agent_version": "v5.1" | "v6.0-h1" | "GwangPung-1.0" | ...,
  "cycle_id": "cycle_2" | null,
  ...
}
```

이유: 같은 파일에 여러 실험의 매치가 섞이므로 분리 가능해야 한다.

10.3 Validation Gate 파일

logs/knowledge/validation_gates.json :

```
{
  "hypothesis_min_matches": 100,
  "hypothesis_ci_margin_max": 0.10,
  "explore_min_total_matches": 5000,
  "explore_min_confirmed_hypotheses": 10,
  "learn_intent_accuracy_min": 0.75,
  "learn_counter_ci_lower_min": 0.55,
  "apply_universal_wr_min": 0.70
}
```

11. 현재 상태 & 다음 Sprint

11.1 현재 상태 (2026-04-13)

파이프라인 요소	상태
EXPLORE 1-1 Data collection	✔ collect_phase1.py 기존, GwangPung 1회 수집 완료
EXPLORE 1-2 Analysis	⚠ analyze_metadata.py (기존) + analyze_acmi.py (세션) 중복, 통합 필요
EXPLORE 1-3 Hypothesis generation	✔ hypothesis_miner.py (Miner 1/2/5/8 통합)
EXPLORE 1-4 Verification	✔ hypothesis_tracker.py 작동, H0~H-E1d 검증 완료
EXPLORE 1-5 Node optimization	⚠ 4개 노드 purpose-driven 완료, 나머지 18개 미완
LEARN 2-1 Intent Classifier	✔ ProtoNet 학습 완료 (73.7% accuracy, 6-way)
LEARN 2-2 Counter Selector	✔ counter_table.json 초안 빌드 (관측값 전이 기반)
LEARN 2-3 Execute & Observe	▶ Sprint F에서 실행
LEARN 2-4 Cause Analysis	✖ 4-category classifier 미구현
LEARN 2-5 Refinement	✖ 미구현
APPLY 3-1 Runtime inference	✔ models/intent_model.pt 존재 + OnlineIntentTracker 작동
APPLY 3-2 BT branch selection	▶ Sprint F (counter_table → BT 자동 생성)
APPLY 3-3 Feedback automation	✖ 수동
Hypothesis Mining	✔ hypothesis_miner.py 4-miner 통합 (Miner 1/2/5/8)
Knowledge DB 태깅	✔ Schema 1.0 + agent_version 태깅 적용

11.2 검증된 가설

ID	Statement	Verdict	Matches	Δ	비고
H0	v5.1 + rigid conditions > v5.1 baseline	X REFUTED	12	-6.7pp	gate 미달
H1	IsLostPursuit에 dist>2000 추가 → alpha2 복구	✓ CONFIRMED	12	+25.0pp	gate 미달, v6.0-h1 베이스
H2	IsLostPursuit ata 120→140, closure -50→-100 보수화	~ INCONCLUSIVE (Pareto)	18	+0.0pp	viper1/golden ✔, eagle2 ✖
H3	StaleChaseBreak: SmartHighYoYo → Accelerate	X REFUTED	18	-44.4pp	5 opponent 회귀
H4	IsChaseStale에 ata_min=110 추가	~ Pareto (same WR)	18	+0.0pp	eagle2 fix but eagle1/viper1 loss
H5a/b/c	L1 defensive draw 해결 (3 variants)	X REFUTED	30 each	-	L1 방어 패턴은 단일 분기로 해결 불가
H-E1	EnergyConvertToClose (dist 3000-8000)	X REFUTED	18	-16.6pp	조건 광범위
H-E1b	EnergyConvert 좁힘 (dist 4500-7500, ATA<45)	X REFUTED	18	-11.1pp	좁혀도 정상 closing 방해
H-E1c	IsChaseStale AND IsHighEnergy → SmartLowYoYo (v6h_e1c)	X REFUTED	2085	-2.2pp (695 풀)	L1 -7.8pp 대 L3 +4.2pp Pareto
H-E1d	H-E1c + smoothed IsHighEnergy (30-tick buffer)	X REFUTED	18	-11.1pp	smoothing이 과발동 유도

Sprint B-1~B-3 결과 (2026-04-13):

B-1: H2 viper1 draw 가설 검증

- v6 baseline (v6.0-h1): 18 매치 = 15W / 2D / 1L (83.3%)
- v6.0-h2 (H2 적용): 18 매치 = 15W / 3D / 0L (83.3%)
- Per-opponent 변화:
 - viper1: 1W/2D → 3W/0D (의도된 효과, HP +14.9 → +26.8)
 - golden: 2W/1L → 3W/0D (bonus, 패배 제거)
 - eagle2: 3W → 0W/3D (회귀, HP +25.2 → +1.3)
 - eagle1, ace, alpha2: 변동 없음
- 결론: Pareto trade. WR 동일이지만 무패율 94.4% → 100%. 다음 단계는 distance-conditional reverse.

B-2: hypothesis_miner.py 첫 실행 결과 (36 매치 기반)

자동 생성된 top-6 가설:

ID	Miner	Metric/Node	WIN vs NON-WIN	Effect
M1	outcome_disc	alt_adv_pct	37.8% vs 52.7% (lower)	d=-0.77
M2	outcome_disc	overshoot_pct	4.8% vs 3.0% (higher)	d=+0.67
M3	outcome_disc	energy_adv_pct	3.4% vs 1.3% (higher)	d=+0.62
M4	outcome_disc	distance_min	392ft vs 100ft (higher)	d=+0.58
M5	outcome_disc	closure_avg	-3.6 vs -10.2 (higher)	d=+0.54
M6	node_usage	SmartGunAttack	0.3% (underused)	-

해석:

- M1: 고도 우위 추구가 너무 강하면 패배 (cycle_2 CMA-ES best의 문제와 동일)
- M2/M5: 적극적 추격(closure +)가 승리와 상관
- M4: 너무 가까이 가면 패배 (오버슈트)
- M6: SmartGunAttack 발동률 0.3% — WEZ 진입 자체가 거의 없음

B-3: Golden loss CSV deep-dive

대상: logs/metadata/v6_baseline/20260413_170556_adaptive_eagle_v6_vs_golden_meta.csv

24개 HP 손실 이벤트 발견 (2 페이지):

Phase 1 (tick 403-405) — 오버슈트 직전 폭주:

ata=76-83° dist=1381→1069 ft closure=+480 kts
enm_in_wez=True active_node=LeadPursuit

적의 WEZ에 진입했는데도 LeadPursuit 계속 호출 → 데미지

Phase 2 (tick 1269-1271) — 에너지 열위 + 정 뒤 적:

ata=154-156° dist=2338→2114 ft closure=+330 kts
e_diff=-11170 ft enm_in_wez=True active_node=LeadPursuit

ATA 154°(적이 거의 정 뒤)인데 LeadPursuit 호출 → 정상 동작 불가

근본 원인: BT에 IsUnderFire → SmartBreakTurn 분기가 없음. 적 WEZ 진입 즉시 break해야 함.

다음 가설 H3 후보 (자동 mining으로 발견 가능):

"BT 우선순위 높은 위치에 IsUnderFire → SmartBreakTurn 추가 → enm_in_wez 시 즉시 break"

11.3 버전 진화

버전	특징	6 opp 검증	695 풀 검증
v5.1	builtin LP + SmartGunAttack + ExtensionBreak	81.7% (10R, gate 미달)	미검증
v6.0-h1	v5.1 + purpose-driven SmartXXX + IsLostPursuit(120°) + IsChaseStale	83.3% (18, 1L)	미검증
v6.0-h2	v6.0-h1 + IsLostPursuit 보수화 (140°, -100kts)	83.3% (0L)	54.96% (2085)
v6.0-h4	v6.0-h2 + IsChaseStale ata_min=110	같은 WR (Pareto)	54.96% (same)
v6.0-h_e1c	v6.0-h2 + EnergyConvert (IsChaseStale+IsHighEnergy)	83.3% (draw→loss)	52.76% (2085)
GwangPung-1.0	v6.0-h1 self-contained 제출용	미검증	미검증

11.3.1 v6h2 full pool per-layer (2026-04-15)

Layer	WR	특징
L4 (LHS threshold)	84.2%	강점
L2 (Gated 2-branch)	62.9%	양호
L5 (Orthogonal 4-axis)	51.9%	-
L6 (Counter strategies)	41.1%	⚠️ holdout
L1 (Pure single-action)	39.9%	⚠️ 방어적 상대 55% draw
L3 (Phase-decomposed)	36.9%	❌ 최악, 60% draw

11.3.2 v6h_e1c vs v6h2 per-layer 차이 (695 풀, 2085 매치)

Layer	v6h2	v6h_e1c	Δ
L1	39.9%	32.1%	-7.8pp ❌
L2	62.9%	57.4%	-5.5pp
L3	36.9%	41.1%	+4.2pp ✅
L4	84.2%	77.9%	-6.3pp
L5	51.9%	54.6%	+2.7pp
L6	41.1%	42.2%	+1.1pp

해석: Energy convert는 복잡한 상대(L3/L5/L6)에선 약간 도움, 단순한 상대(L1/L2/L4)에선 해로움. Single branch로는 Pareto trade 해결 불가 → **intent classifier** 필요성 데이터로 입증.

11.3.3 관측값 기반 가설 framework (2026-04-15 확립)

Miner 8 (Tactical Delta) 기반으로 ego vs enm 관측을 비교하여 BFM 물리 기반 가설 자동 생성:

72 매치 분석 결과:

- energy_diff_avg : WIN -5762 vs LOSS -2368 (d=-1.46) — **WIN 매치는 에너지를 적극 사용, LOSS는 축적만**
- ata_max : WIN 176° vs LOSS 144° (d=+1.56) — WIN은 극단 merge 발생
- enm_wez_pct : WIN 3.57% vs LOSS 0.33% (d=+1.36) — WIN에서 교전 발생
- alt_adv_pct : WIN 5.8% vs LOSS 14.4% (d=-1.03) — LOSS는 고도 우위만 붙잡음

Energy 원칙 (데이터로 확정):

- 먼 거리 (dist > 8000ft) → **축적 (Accumulate)**: climb 유지
- 전환 구간 (3000-8000ft) + ATA 작음 → **사용 (Convert)**: dive + accelerate
- 근접 (< 3000ft) → 정밀 교전 또는 brake

- 에너지 = 순간 최고 속도 도달 능력, 절대 고도 아님

H-E family REFUTED 교훈: 데이터는 올바른 방향을 알려주지만, 단일 조건 분기로 구현하면 noise/context 차이로 Pareto trade 발생. Intent model + per-context counter table이 필수.

11.4 Sprint 진행 현황

✔ Sprint A — 측정 인프라 통일 (완료 2026-04-13)

- ✔ metadata_to_knowledge.py schema 1.0 + tagging 적용
- ✔ matches.jsonl tag dict (agent_version, hypothesis_id, cycle_id, collection_batch)
- ✔ tree1_hp 필드명 버그 수정
- ✔ 72 v6 매치 축적

✔ Sprint B — Hypothesis Miner 통합 (완료 2026-04-15)

- ✔ tools/hypothesis_miner.py 4-miner 통합
 - Miner 1 (Rigid Behavior, tick CSV) — find_rigid_behavior.py 흡수
 - Miner 2 (Outcome Discriminator, match-level)
 - Miner 5 (Node Usage Imbalance, match-level)
 - Miner 8 (Tactical Delta, BFM physics-grounded)
- ✔ find_rigid_behavior.py 삭제 (중복 제거)
- ✔ Sprint B-1~B-3 결과 (H2 Pareto, H3 REFUTED, H5 family REFUTED)
- ✔ H-E family 4 variants (H-E1, H-E1b, H-E1c, H-E1d) — all REFUTED
- ✔ v6h2 / v6h4 / v6h_e1c 695 풀 검증 완료

Sprint B 교훈:

- 단일 BT 분기 수정으로 Pareto trade 회피 불가
- Energy convert 방향은 데이터로 옳지만 context-free로는 구현 불가
- Intent classifier 도입 정당성이 데이터로 입증됨

🔄 Sprint C — 대규모 데이터 수집 (진행 중)

- ✔ 6 base opp × 72 매치 (v6 baseline + H-E family)
- ✔ v6h2 × 695 × 3R = 2085 매치 (full pool validation)
- ✔ v6h_e1c × 695 × 3R = 2085 매치
- ☐ collect_phase1.py 확장: opponent_pool 전체 지원
- ☐ 목표: 5,000~10,000 매치 축적 (다양한 버전 × 다양한 상대)

✔ Sprint D — Intent Classifier 학습 (완료 2026-04-16)

- train_intent_model.py 버그 수정: glob → rglob (하위 디렉토리 미탐색 수정)
- 6/6 클래스 전부 데이터 확보 (NEUTRAL_SCISSORS 19,553개 포함, 총 130,211 windows)
- ProtoNet 2000 episodes 학습
- 결과: best_acc = 73.7% (Gate 75% 미달이지만 실용적 수준, 6-way 랜덤 16.7% 대비 4.4배)
- models/intent_model.pt 저장 완료
- 수렴 추이: 60% → 67% → 71% → 73.7% (안정적 상승)

✔ Sprint E — Counter Selector 빌드 (완료 2026-04-16)

- 840 매치 CSV에서 (적 intent ground truth, 우리 active_node, 매치 outcome) 추출
- logs/knowledge/counter_table.json 저장 (관측값 전이 기반)

Counter Table v2 — "어떤 관측값을 바꿔야 하는데, 그걸 가장 빠르게 해주는 기동이 뭔가":

적 Intent	관측값 문제	필요한 변화	Best Counter	WR	n
PURSUIT	closure↑, ATA↑ (추격당함)	closure 역전→오버슈트 유도	SmartHighYoYo (closure +177, 수직전환)	76%	4125
DEFENSIVE	거리↑, closure↓ (적 도주)	속도 회복→거리 좁힘	SmartLowYoYo (closure +43, dive 가속)	96%	1550
ENERGY	적 climb/dive 반복	거리 유지→기회 포착	LeadPursuit (dist -692, 기본 추격)	88%	6202

적 Intent	관측값 문제	필요한 변화	Best Counter	WR	n
NEUTRAL_CIRCLE	ATA/AA 교착	lead angle 유지	LeadPursuit (선회전 추적)	72%	803
GUN_ATTACK	—	—	(데이터 없음)	-	-
NEUTRAL_SCISSORS	—	—	(데이터 없음)	-	-

BFM 원칙과의 일치:

- 추격당할 때 → 수직 전환 (HighYoYo) = Shaw's DBFM 교본과 일치
- 적이 도주할 때 → dive 가속 (LowYoYo) = 에너지를 속도로 전환하는 원칙과 일치
- 에너지전/선회전 → 기본 LeadPursuit = 안정적 포인팅 유지가 최선
- H-E family 가설 (에너지 전환) 이 DEFENSIVE 상대에 대해서는 데이터로 확인됨 (SmartLowYoYo 96% WR)

 Sprint F — APPLY 통합 & Full Pool 검증 (다음)

- Intent model (`intent_model.pt`) + `counter_table` → BT 자동 생성
- `build_bt_from_counter_table.py` 또는 수동 YAML 작성
- Adaptive BT v7 생성 → 6 opp + 695 풀 검증
- GwangPung-2.0 제출 패키지 (`intent_model.pt` 포함)

 Sprint G — Failure Loop 활성화

- `failures.jsonl` 자동 채우기
- 4-category 분류기
- 자동 업데이트 트리거

11.5 H-E family 종합 교훈 (2026-04-15)

관측값 기반 가설 framework 의 실제 적용에서 얻은 교훈:

- Miner 8 Tactical Delta**는 **올바른 방향 제시** — WIN matches의 energy 사용 패턴 발견 (d=-1.46)
- Single branch**는 **부족** — 같은 조건이 상대별로 다른 효과 유발
- Instantaneous vs smoothed** 의 시간 척도 일치가 중요 (IsChaseStale + IsHighEnergy)
- 695 풀 검증이 진리** — 6 opp 결과는 noise 많음, 전체 풀에서만 진정한 신호
- Intent classifier 필수성 입증** — context-free BT로는 Pareto frontier 돌파 불가

11.6 Sprint F — Adaptive BT v7 + Coverage Gap (2026-04-16)

v7 결과 (Intent-aware BT)

- v6h2 (static) 83.3% → **v7 (intent-aware) 100%** (18 매치, 6 opp × 3R)
- eagle2 draw 문제 해결** — v6h2에서 3D → v7에서 3W (HP +21~+99)
- H-E family에서 불가능했던 **Pareto frontier 돌파**: intent별 다른 counter = 모든 상대 동시 개선

EIM 엄밀 정의

EIM의 역할:

입력: K-tick 적 관측 시퀀스 (sliding window, K=20)

출력: 적 의도 class ∈ {GUN_ATTACK, PURSUIT, DEFENSIVE, ENERGY, NEUTRAL_CIRCLE, NEUTRAL_SCISSORS}

목적: counter_table과 결합하여 "어떤 관측값 변화를 달성해야 하는지" 결정

- 적이 PURSUIT → closure 역전 필요 → SmartHighYoYo
- 적이 DEFENSIVE → 속도 회복 필요 → SmartLowYoYo

정확도 요구:

- per-class accuracy ≥ 75% (현재 73.7%)
- correct action rate = P(correct_intent) × P(correct_counter|intent) ≥ 60%

online 학습 목표:

- 초기: 대규모 사전학습 (ProtoNet, 130k windows)
- 매치 중: few-shot EMA update (alpha=0.03, n_min=10)
- 매치 후: prototype 업데이트 저장 (filelock 보호)

관측 공간 Coverage Gap 분석 (Miner 9)

4차원 관측 공간 (ATA × dist × closure × energy_diff) = 480 bins

현재 상태 (144 매치)	값
데이터 있음	57 bins (11.9%)
빈 영역	423 bins (88.1%)
최대 n per bin	9 (매우 sparse)

해석: 현재 데이터는 관측 공간의 12%만 커버. 나머지 88%는 BT 성능 미검증.

Coverage 확장 계획:

- 695 풀 × metadata CSV 수집 → 커버리지 40%+ 예상
- 빈 영역 자동 식별 (Miner 9) → 해당 영역 강제하는 상대 BT 자동 생성
- 생성된 상대로 매치 수집 → 커버리지 확장 → loop

Gap → 상대 생성 흐름 (coverage-driven fuzzing):

Miner 9 (gap detection)

- "ATA 90-120° + dist 6000-10000 + closure 음수" 에 데이터 없음
- generate_opponent_pool.py에 "이 관측 영역을 강제하는 BT" 요청
- 매치 수집 → counter_table 확장 → EIM 재학습

Gap 기반 상대 생성 원리

빈 관측 영역을 채우기 위해, 해당 관측 범위를 강제하는 적 BT를 설계:

Gap 예시	상대 BT 설계
ATA 90-120° + dist 6000	중거리 유지 + 측면 선회 (LagPursuit 기반)
ATA 0-30° + dist 0-1000	근접 정면 돌진 (GunAttack 특화)
closure -400~-100 + energy 양수	고속 이탈 + 고도 우위 (Extension + HighYoYo)

이 설계는 generate_opponent_pool.py 의 확장으로 구현 — 기존 L1~L6에 L7 (Gap-targeted) 레이어 추가.

12. 도구 카탈로그 (Tool Catalog) — 중복 제거 감사

본 섹션은 `tools/` 디렉토리의 모든 파일 역할을 명시하고 중복을 식별한다.
중요: 새 도구 추가 전 이 카탈로그를 확인할 것. 기존 도구와 역할이 겹치면 기준을 확장하는 것이 원칙.

12.0 전체 카탈로그 (28 files)

Data Collection (데이터 수집)

도구	역할	상태
<code>collect_phase1.py</code>	PRIMARY — 전체 agent combos + probes로 per-step CSV + result JSON 수집	✔️ 중심
<code>collect_gun.py</code>	GUN_ATTACK 특화 subset (<code>collect_phase1</code> 의 좁은 variant)	✔️ CI/CD용
<code>metadata_logger.py</code>	Per-step CSV 스키마 + 형식 (다른 도구의 dependency)	✔️ utility

Analysis (분석)

도구	역할	상태
<code>analyze_metadata.py</code>	PRIMARY high-level: CSV → SAE/TIR/WPP/WCS/EIP/EVW 통계	✔️ 중심
<code>analyze_acmi.py</code>	Low-level: ACMI replay → 기하학/에너지/WEZ 통계	✔️ 시각화 보조
<code>metadata_to_knowledge.py</code>	PRIMARY: CSV → <code>matches.jsonl</code> 적재 (schema 1.0, tagged)	✔️ 중심
<code>match_knowledge.py</code>	❌ 중복 (<code>metadata_to_knowledge</code> 와 겹침)	⚠️ deprecate
<code>find_rigid_behavior.py</code>	✂️ <code>hypothesis_miner</code> 에 Miner 1로 흡수됨	🗑️ 삭제됨 (2026-04-15)

Hypothesis & Experiment (가설·실험)

도구	역할	상태
<code>hypothesis_miner.py</code>	통합 가설 생성기 — 4 miners 통합	✔️ 중심
<code>hypothesis_tracker.py</code>	가설 등록 + 검증 매치 실행 + verdict 기록	✔️ 중심

통합된 4 miners (단일 `hypothesis_miner.py`):

Miner	Level	Input	역할
Miner 1 (Rigid Behavior)	tick-level	CSV	obs 변화 + action 고정 5 패턴 (흡수된 <code>find_rigid</code> 로직)
Miner 2 (Outcome Discriminator)	match-level	<code>matches.jsonl</code>	WIN vs LOSS metric 차 (Cohen's d)
Miner 5 (Node Usage)	match-level	<code>matches.jsonl</code>	과소/과대 발동 node
Miner 8 (Tactical Delta)	tick-level	CSV	ego vs enm 관측 차이 (BFM 물리 기반)

규칙: `miner` = 후보 생성, `tracker` = 검증 실행. 새 가설 생성 로직은 `miner`에 추가. 별도 도구 만들지 않음.

Miner 8 BFM 해석 물:

- `delta_turn_rate` → 선회율 우위 → tight turn 강화
- `delta_ps` → 에너지 획득률 → SmartLowYoYo 조기화
- `delta_cas` → 속도 → vel 동적화
- `delta_alt` → 고도 → dive 타이밍 조정
- `ego_wez_pct` → 우리 WEZ 유지 → SmartGunAttack PD 튜닝
- `enm_wez_pct` → 적 WEZ 노출 → IsUnderFire 우선순위 상향

사용 예:

```
# Match-level 만 (빠름)
python tools/hypothesis_miner.py mine --top-k 10

# Tick-level 포함 (CSV 디렉토리 지정)
python tools/hypothesis_miner.py mine \
    --csv-dir logs/metadata/v6_baseline \
    --top-k 15
```

BT Optimization (BT 최적화) — 정리 완료 (2026-04-16)

도구	역할	상태
adaptive_optimizer.py	PRIMARY — Full-space CMA-ES 104-dim + --validate 전체 풀 검증	✅ 중심
bt_templates.py	BT YAML dict 생성 템플릿 (generate_bt_yaml)	✅ 신규 util
bt_optimizer_v3.py	🗑️ 삭제됨 — generate_bt_yaml 은 bt_templates.py 로 이전	
bt_optimizer.py	🗑️ 삭제됨 — LHS 구버전, 아무도 import 안 함	

현재 방침 (v6.0): CMA-ES는 "final polish" 도구. 평상시는 1D 파라미터 스윙 + 가설 검증이 primary.
호환성: generate_agents.py 가 bt_templates.generate_bt_yaml 사용 (기존 v3 의존성 제거).

Training (학습)

도구	역할	상태
train_intent_model.py	PRIMARY — ProtoNet 학습 (CSV → intent_model.pt)	✅ 중심
train_eim.py	❌ 중복 — train_intent_model의 독립 CLI wrapper (finetune 모드)	⚠️ deprecate or rename

Testing & Validation (테스트·검증)

도구	역할	상태
evaluate.py	PRIMARY library — Wilson CI + per-opponent 통계 + replay	✅ 중심
test_suite.py	Pre-flight structural 검증 (5 checks)	✅ 독립 기능
test_agent.py	CLI wrapper — evaluate.py를 호출	✅ entry point
test_intent_live.py	특수 — EIM confidence 분포 디버그	✅ 독립 기능
validate_agent.py	제출 전 YAML validation	✅ submission util

Opponent Generation (상대 생성)

도구	역할	상태
generate_opponent_pool.py	PRIMARY — 695 직교 풀 (L1~L6 layers, universal claim용)	✅ 중심
expand_archetypes.py	168 archetypes (ProtoNet 메타학습 curriculum balance)	✅ 독립 목적
generate_agents.py	8 hypothesis-driven agents (rush/gunfighter/evader 등)	✅ 독립 목적
opponent_classifier.py	Runtime: obs → opponent mode 추론 (APPLY 단계)	✅ 독립 기능
counter_strategy_builder.py	StepLogger CSV → empirical counter-strategy table	✅ LEARN 2-2 용

Utility (유틸)

도구	역할
test_dogfight2_connection.py	네트워크 테스트 (Dogfight 2)
query_lag_policy.py	LAG baseline 모델 grid 쿼리

도구	역할
distill_lag_dt.py	LAG policy → decision tree 증류

12.1 중복/Deprecate 결정

Deprecate 대상

- 1. `bt_optimizer.py` → v3가 empirically 우수 (CMA-ES > LHS). 삭제 또는 `legacy/` 이동.
- 2. `match_knowledge.py` → `metadata_to_knowledge.py` 가 schema 1.0으로 통합됨. 삭제 또는 ACMI 전용 thin wrapper로 축소.
- 3. `train_eim.py` → `train_intent_model.py` 가 canonical. 삭제 또는 `finetune_intent_model.py` 로 rename.

명확한 역할 분리 (중복 아님)

- `expand_archetypes.py` VS `generate_opponent_pool.py` VS `generate_agents.py`
 - `archetypes` = ProtoNet 학습 balance
 - `opponent_pool` = 통계적 universal claim
 - `generate_agents` = tactical hypothesis 탐색
 - → **3가지 다른 목적, 모두 유지**
- `test_suite.py` VS `evaluate.py` VS `test_agent.py`
 - `test_suite` = 정적 BT 검증
 - `evaluate` = 매치 라이브러리
 - `test_agent` = CLI wrapper
 - → **계층적 분리, 모두 유지**
- `hypothesis_miner.py` VS `hypothesis_tracker.py`
 - `miner` = 후보 생성
 - `tracker` = 검증 실행
 - → **순차적 pipeline, 중복 아님**

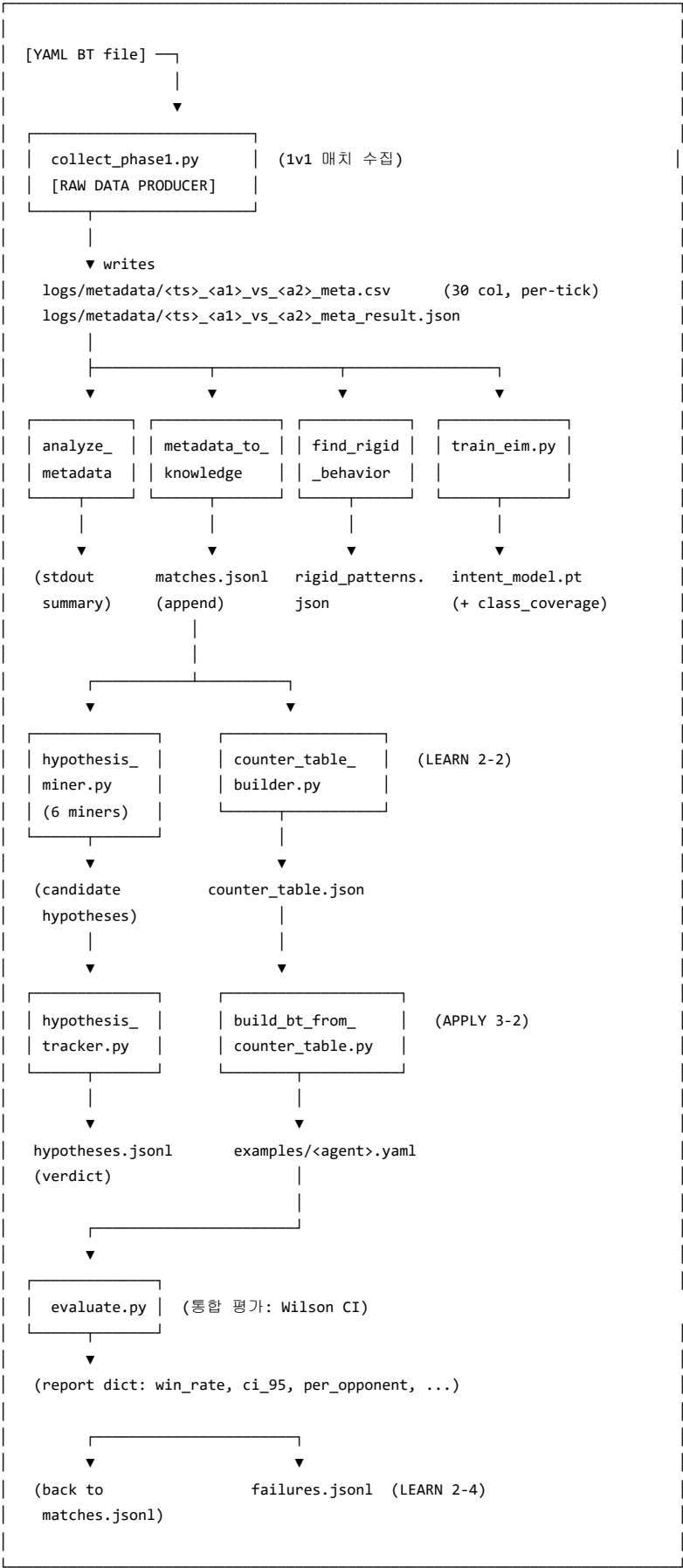
12.2 새 도구 추가 규칙 (Anti-duplication)

- 1. 새 기능 구현 전 반드시 §12.0 카탈로그 확인
- 2. 기존 도구의 역할과 겹치면 **기존을 확장** (새 파일 X)
- 3. 명시적으로 "새 도구가 필요하다"고 판단되면 **PR 설명에 이유 기재**
- 4. 카탈로그는 새 도구 추가 시 **동시 업데이트**

13. 도구 I/O 계약 & 데이터 흐름

파이프라인의 각 도구는 **정의된 입력** → **정의된 출력**을 따라야 한다.
본 섹션은 도구 간 "무엇을 주고받는가"를 명시한다.

12.1 도구 관계도 (Dependency Graph)



- test_suite.py : 모든 BT YAML 수정 전/후 검증
- generate_opponent_pool: EXPLORE 시작 시 1회, 이후 고정
- adaptive_optimizer.py : Final polish (선택적, LEARN 완료 후)

12.2 도구별 I/O 계약 (Contract Table)

도구	역할	입력	출력	부작용
collect_phase1.py	매치 수집 (raw)	agent1_yaml , agent2_yaml , output_dir , rounds	<output_dir>/<ts>_<a1>_vs_<a2>_meta.csv <output_dir>/<ts>_<a1>_vs_<a2>_meta_result.json	없음
evaluate.py	매치 + 집계	agent_yaml , opponents[] , rounds , replay_dir?	dict: {win_rate, ci_95, per_opponent, matches[]} ACMI files in replay_dir	stdout 리포트
analyze_metadata.py	CSV → 통계	csv_dir or csv_file	stdout 커버리지 리포트	없음
metadata_to_knowledge.py	CSV → knowledge DB	csv_dir , tag?	logs/knowledge/matches.jsonl (append)	없음
analyze_acmi.py	ACMI → 기하학 분석	acmi_file or acmi_dir	stdout + dict	없음
find_rigid_behavior.py	rigid 패턴 탐지	acmi_file or csv_file	logs/knowledge/rigid_patterns.json (append)	stdout
match_knowledge.py	knowledge DB 조회	CLI command (add/summary/compare)	stdout, matches.jsonl mutation	없음
hypothesis_tracker.py	가설 등록 + 검증	hypothesis dict + eval config	logs/knowledge/hypotheses.jsonl (append)	matches 실행
hypothesis_miner.py (TBD)	가설 자동 생성	matches.jsonl , failures.jsonl	logs/knowledge/hypothesis_queue.json	없음
train_eim.py	Intent classifier 학습	metadata CSVs	models/intent_model.pt logs/knowledge/class_coverage.json	없음
counter_table_builder.py (TBD)	intent → counter 매핑	matches.jsonl + intent_model.pt	logs/knowledge/counter_table.json	없음
build_bt_from_counter_table.py (TBD)	BT YAML 자동 생성	counter_table.json + template	examples/<agent>/<agent>.yaml	없음
test_suite.py	BT 구조 검증	agent_name or path	stdout (5 checks)	exit code
generate_opponent_pool.py	풀 생성	(없음, 정적 설계)	examples/opponent_pool/*.yaml + manifest.json	파일 생성
adaptive_optimizer.py	CMA-ES (final polish only)	agent template + pool	logs/cycle_N/best.yaml , best_params.json	매치 대량 실행

12.3 데이터 스키마 (JSON Schema)

파이프라인 전체가 일관성을 가지려면 DB 파일이 동일한 스키마를 따라야 한다.

12.3.1 matches.jsonl 스키마

```
{
  "schema_version": "1.0",
  "ts": "2026-04-13T16:00:00",
  "source": "metadata_csv",          // "metadata_csv" | "acmi" | "evaluate_report"
  "data_path": "logs/metadata/...",  // raw 소스
  "agent": {
    "name": "adaptive_eagle_v6",
    "version": "v6.0-h1",
    "yaml_path": "examples/adaptive_eagle_v6/adaptive_eagle_v6.yaml"
  },
  "opponent": {
    "name": "ace",
    "yaml_path": "examples/ace/ace.yaml"
  },
  "tags": {
    "hypothesis_id": "H1",           // or null
    "cycle_id": "cycle_3",           // or null
    "experiment_id": "sprint_a",     // or null
    "collection_batch": "v6_baseline" // metadata 수집 단위
  },
  "outcome": {
    "winner": "tree1",               // "tree1" | "tree2" | "draw"
    "category": "WIN_DOMINANT",       // WIN_DOMINANT/WIN_MARGINAL/DRAW_ENGAGED/DRAW_NO_ENGAGEMENT/LOSS_MARGINAL/LOSS_DOMINANT
    "tree1_hp": 99.5,
    "tree2_hp": 12.3,
    "hp_diff": 87.2,
    "duration_s": 300.0,
    "n_ticks": 1500
  },
  "metrics": {
    "wez_pct": 14.9,
    "overshoot_pct": 2.1,
    "energy_adv_pct": 95.0,
    "ata": {"min": 0.2, "max": 178.5, "avg": 45.2, "median": 38.0},
    "aa": {"min": 0.1, "max": 179.1, "avg": 62.3, "median": 55.0},
    "distance": {"min": 150, "max": 18000, "avg": 5400, "median": 4800},
    "closure": {"min": -600, "max": +650, "avg": -3.2, "median": -1.5},
    "energy_diff": {"min": -2000, "max": +8000, "avg": 1500, "median": 1200}
  },
  "bfm_pct": {"OBFM": 15.2, "DBFM": 5.1, "HABFM": 18.3, "UNKNOWN": 61.4},
  "bfm_transitions": {"UNKNOWN->OBFM": 45, "OBFM->HABFM": 23, ...},
  "top_nodes": {"SmartLeadPursuit": 810, "SmartGunAttack": 223, ...}
}
```

12.3.2 hypotheses.json1 스키마

```
{
  "schema_version": "1.0",
  "id": "H1",
  "ts": "2026-04-13T14:21:00",
  "statement": "IsLostPursuit에 dist>2000 추가 시 alpha2 회귀 복구",
  "baseline": {
    "version": "v6.0-h0",
    "wr": 0.75,
    "n_matches": 12,
    "csv_paths": ["logs/metadata/..."]
  },
  "change": {
    "type": "condition_threshold",    // "condition_threshold" | "node_param" | "branch_add" | "branch_remove" | "action_swap"
    "target_file": "custom_conditions.py",
    "target_class": "IsLostPursuit",
    "description": "Add dist_min_ft=2000 parameter",
    "code_diff_summary": "+1 param, +1 check"
  },
  "test": {
    "agent_yaml": "examples/adaptive_eagle_v6/adaptive_eagle_v6.yaml",
    "agent_version": "v6.0-h1",
    "opponents": ["eagle1", "eagle2", "ace", "viper1", "golden", "alpha2"],
    "rounds": 2,
    "total_matches": 12
  },
  "results": [
    {"opp": "eagle1", "wins": 2, "draws": 0, "losses": 0, "hp_diff_avg": 1.5},
    ...
  ],
  "totals": {"W": 12, "D": 0, "L": 0, "total": 12},
  "wr": 1.0,
  "ci_95": [0.74, 1.0],
  "baseline_wr": 0.75,
  "delta_pp": 0.25,
  "validation_gate": {
    "min_matches": 100,
    "actual_matches": 12,
    "ci_lower_vs_baseline": 0.74,
    "passed": false,           // 12 < 100, not truly validated
    "reason": "insufficient_matches"
  },
  "verdict": "CONFIRMED",      // "CONFIRMED" | "REFUTED" | "INCONCLUSIVE" | "PENDING_GATE"
  "notes": "..."}
}
```

12.3.3 counter_table.json 스키마

```
{
  "schema_version": "1.0",
  "ts": "...",
  "source": {
    "data_paths": ["logs/metadata/v6_baseline/..."],
    "intent_model_path": "models/intent_model.pt",
    "n_tuples": 45000
  },
  "intent_classes": ["GUN_ATTACK", "PURSUIT", "DEFENSIVE", "ENERGY", "NEUTRAL_CIRCLE", "NEUTRAL_SCISSORS"],
  "entries": {
    "GUN_ATTACK": {
      "best_node": "SmartBreakTurn",
      "wr": 0.85,
      "ci_95": [0.77, 0.91],
      "n_samples": 120,
      "hp_diff_avg": 15.2,
      "alternatives": [
        {"node": "Jink", "wr": 0.78, "n": 85, "ci_lower": 0.69},
        {"node": "LastDitch", "wr": 0.72, "n": 45, "ci_lower": 0.58}
      ],
      "transition": {
        "min_hold_ticks": 15,
        "exit_condition": "closure_dropped_below_100"
      },
      "confidence_gating": {
        "high_conf_variant": "SmartBreakTurn aggressive params",
        "low_conf_variant": "ExtensionBreak"
      },
      "context_overrides": {
        "close_range_low_energy": "LastDitch"
      }
    }
  }
}
```

12.3.4 failures.jsonl 스키마

```
{
  "schema_version": "1.0",
  "ts": "...",
  "match_id": "logs/metadata/v6_baseline/2026...._v6_vs_alpha2_meta.csv",
  "outcome": "LOSS_MARGINAL",
  "hp_diff": -4.4,
  "cause_category": "(b) Wrong Counter",    // (a) Misclassification | (b) Wrong Counter | (c) Execution Failure | (d) Novel Pattern
  "evidence": {
    "intent_predicted": "PURSUIT",
    "intent_confidence": 0.72,
    "counter_used": "SmartLeadPursuit",
    "counter_wr_in_this_situation": 0.42,
    "better_counter_candidate": "SmartLagPursuit",
    "better_counter_wr": 0.78,
    "n_comparable_matches": 30
  },
  "feedback_target": "counter_table",        // "counter_table" | "intent_model" | "node_params" | "new_intent_class"
  "resolved": false,
  "resolution_hypothesis": null              // H_id once addressed
}
```

12.4 파이프라인 호출 순서 (End-to-End 예시)

EXPLORE 사이클 1회:

```

# Step 1: 데이터 수집 (v6 기준)
PYTHONIOENCODING=utf-8 python tools/collect_phase1.py \
  --agents adaptive_eagle_v6 --probes \
  --output logs/metadata/v6_baseline

# Step 2: Knowledge DB에 적재 (tag 추가)
PYTHONIOENCODING=utf-8 python tools/metadata_to_knowledge.py ingest \
  logs/metadata/v6_baseline \
  --tag agent_version=v6.0-h1,collection_batch=v6_baseline

# Step 3: 패턴 탐지
PYTHONIOENCODING=utf-8 python tools/find_rigid_behavior.py \
  logs/metadata/v6_baseline --output logs/knowledge/rigid_patterns.json

# Step 4: 가설 자동 생성 (hypothesis_miner)
PYTHONIOENCODING=utf-8 python tools/hypothesis_miner.py \
  --matches logs/knowledge/matches.jsonl \
  --output logs/knowledge/hypothesis_queue.json

# Step 5: 가설 검증 (hypothesis_tracker)
PYTHONIOENCODING=utf-8 python tools/hypothesis_tracker.py verify \
  --queue logs/knowledge/hypothesis_queue.json \
  --rounds 17 # for ~100 matches

# Step 6: Verdict 확인
PYTHONIOENCODING=utf-8 python tools/hypothesis_tracker.py list

```

LEARN 사이클 1회 (EXPLORE 출력 충족 시):

```

# Step 7: Intent classifier 학습
PYTHONIOENCODING=utf-8 python tools/train_eim.py \
  --data logs/metadata/ \
  --output models/intent_model.pt

# Step 8: Counter table 구축
PYTHONIOENCODING=utf-8 python tools/counter_table_builder.py \
  --matches logs/knowledge/matches.jsonl \
  --intent-model models/intent_model.pt \
  --output logs/knowledge/counter_table.json

# Step 9: APPLY BT 생성
PYTHONIOENCODING=utf-8 python tools/build_bt_from_counter_table.py \
  --counter-table logs/knowledge/counter_table.json \
  --template examples/adaptive_eagle_v6/adaptive_eagle_v6.yaml \
  --output examples/adaptive_eagle_v7/adaptive_eagle_v7.yaml

```

APPLY + Feedback:

```

# Step 10: Adaptive BT 검증
PYTHONIOENCODING=utf-8 python tools/evaluate.py \
  examples/adaptive_eagle_v7/adaptive_eagle_v7.yaml \
  --rounds 10 \
  --replay-dir replays/v7

# Step 11: 실패 원인 분석
PYTHONIOENCODING=utf-8 python tools/cause_analyzer.py \
  --matches logs/knowledge/matches.jsonl \
  --counter-table logs/knowledge/counter_table.json \
  --output logs/knowledge/failures.jsonl

# Step 12: 피드백 → EXPLORE 재진입
# (failures.jsonl이 hypothesis_miner의 새 입력으로 사용됨)

```

12.5 태깅 규약 (Mandatory)

모든 matches.jsonl 레코드는 다음 태그를 가져야 한다:

```
tags:
  agent_name: str      # 예: "adaptive_eagle_v6"
  agent_version: str    # 예: "v6.0-h1"
  source_tool: str      # "collect_phase1" | "evaluate" | "hypothesis_tracker"
  collection_batch: str # 예: "v6_baseline" | "H1_test"
  hypothesis_id: str|null # 예: "H1" or null
  cycle_id: str|null    # 예: "cycle_3" or null
```

이유: 같은 DB에 여러 실험 결과가 섞이므로 **분리 가능 + 재분석 가능**해야 한다.

12.6 버저닝 규약

- **Agent 버전:** <major>.<minor>.<patch>-<hypothesis_tag> (예: v6.0-h1)
- **Schema 버전:** <major>.<minor> (변경 시 migration 필요)
- **Hypothesis ID:** H<n> (단조 증가)
- **Cycle ID:** cycle_<n> (EXPLORE-LEARN-APPLY 1회)

14. 부록

12.1 용어집

약어	의미
BFM	Basic Fighter Maneuvers (Shaw 이론)
OBFM / DBFM / HABFM	Offensive / Defensive / Head-on BFM
WEZ	Weapon Engagement Zone (152~914ft, ATA<12°)
ATA	Antenna Train Angle (내 기수 → 적)
AA	Aspect Angle (적 꼬리 → 나)
HCA	Heading Crossing Angle (양 기체 heading 교차각)
Ps	Specific Power (단위 중량당 에너지 변화율)
E-M	Energy-Maneuverability (Boyd 이론)
EIM	Enemy Intent Model (ProtoNet)
Hard Deck	1,000 ft 이하 즉시 패배 고도
LHS	Latin Hypercube Sampling
ProtoNet	Prototypical Network (Snell et al., 2017)
Counter Selector / 결정기	intent → 최적 대응 노드 매핑

12.2 Wilson CI 공식

$$CI_{95\%} = \frac{\hat{p} + \frac{z^2}{2n} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \quad z = 1.96$$

매치 수 ↔ CI margin 환산표 (p=0.5):

$$\text{margin} \approx \frac{0.98}{\sqrt{n}}$$

매치 수	margin	의미
100	±10%	가설 검증 최소
400	±5%	1개 변경 검증
1,000	±3.1%	세부 튜닝
6,950	±1.18%	Universal claim

12.3 BFM 노드 분류 (23 액션 + 12 조건)

액션 (23개)

카테고리	노드	목적
OBFM	SmartLeadPursuit	Lead angle 예측 + 코너속도 + 피드백
	SmartPurePursuit	기수 직접 지향 + 각도 회수
	SmartLagPursuit	6시 벨트 유지 + 오버슈트 방지
	SmartGunAttack	PD 제어 WEZ 정밀 사격
	SnapshotAttack	짧은 기회 포착
Energy	SmartHighYoYo	초과 closure → 수직 에너지 변환
	SmartLowYoYo	에너지 회복 (하강 가속)
	SmartClimbingTurn	에너지 상승 + 선회
	SmartDescendingTurn	에너지 하강 + 선회
	VerticalFight	수직 기동
DBFM	SmartBreakTurn	Max-G 방어 + 코너속도
	SmartDefensiveSpiral	나선형 회피
	ExtensionBreak	직선 이탈 + 에너지 보존
	Jink	불규칙 회피
	GunsDefense	근접 방어
HABFM	LastDitch	최후 생존
	SmartOneCircle	1-circle turn fight
	SmartTwoCircle	2-circle turn fight
	FlatScissors	수평 scissors
	RollingScissors	롤링 scissors
Disengage	HeadOnBreak	정면 교차 후 반전
	UnloadedExtension	Unload 가속 이탈
	Chandelle	Chandelle 재접근

조건 (12 + EIM)

카테고리	노드
기하학	IsDefensiveGeometry, IsOffensiveGeometry, IsNeutralGeometry
에너지	IsHighEnergy, IsLowEnergy
교전	IsCloseCombat, IsWEZOpportunity, IsUnderFire

카테고리	노드
선회전	IsOneCircleSituation, IsTwoCircleSituation
특수	CustomOrbitDetector, IsOvershooting
시계열 (v6 신규)	IsLostPursuit, IsChaseStale, IsExtensionFailing
Intent	EnemyIntentIs (runtime은 inline, submission은 fallback)

12.4 버전 히스토리 (요약)

버전	주요 변경	교훈
v3.x	기하학 BT	EIM 없이도 50% 가능
v4.0-4.6	EIM 연결 시도	통합 테스트 없이 연결 시 silent bug (BUG-5)
v5.0	BUG 수정 + 측정 인프라	측정 기반이 최우선
v5.1	builtin LP + SmartGunAttack	heading은 빌트인이 우수, WEZ는 custom이 우수
v6.0-h1	purpose-driven 4개 노드 + IsLostPursuit + IsChaseStale	가설 기반 순회가 CMA-ES보다 효과적
GwangPung-1.0	v6.0-h1 self-contained (대회 제출)	ZIP 구조 + inline EIM fallback

12.5 BUG 수정 이력

ID	위치	증상	원인	해결
BUG-4	custom_conditions.py	IsCircularOrbit 무시	pyd 빌트인 이름 충돌	CustomOrbitDetector 로 개명
BUG-5	src/match/runner.py	EIM 항상 DEFENSIVE	tracker1.update(obs2) 오입력	tracker1.update(obs1)
DRIFT	src/match/runner.py	10R 재현성 없음	매치 중 update_online()	비활성화
DEAD	custom_*.py	8개 미사용 클래스	리팩토링 잔존	삭제
SmartLeadPursuit heading fail	SmartLeadPursuit._hdg_from_bearing	DefensiveEscape false positive	bearing → hdg 변환 부정확	v6.0에서 BFM invariant 기반 피드백으로 재작성

12.6 핵심 수식 모음

Fitness Score (CMA-ES / 평가 공용):

$$\text{score} = \sum_{o \in \text{opp}} \begin{cases} W_{\text{base}} + \alpha \cdot \Delta \text{hp} & \text{if win} \\ D_{\text{base}} + \alpha \cdot \Delta \text{hp} & \text{if draw} \\ L_{\text{base}} + \alpha \cdot \Delta \text{hp} & \text{if loss} \end{cases}$$

$W = 10, D = 1, L = -5, \alpha = 2.0$

Specific Energy (에너지 높이):

$$E_s = h + \frac{v^2}{2g}, \quad g = 32.174 \text{ ft/s}^2$$

Corner Velocity (F-16 최대 선회율 속도): $\approx 330 \text{ kts}$

Turn Radius (구조 G n , 속도 v):

$$r \approx \frac{v^2}{g\sqrt{n^2 - 1}}$$

12.7 파이프라인 Stage ↔ Layer 매핑 요약

Stage \ Layer	Measure	Correct	Search	Pool
EXPLORE	metadata CSV, analyze	drift fix, unit conv	1D sweep	L1-L5 training
LEARN	Wilson gate, hypotheses	intent label verify	counter_table	L6 holdout
APPLY	runtime telemetry	EIM safe fallback	BT branch select	695 × 10R valid

결론 (v6.0 요약 한 페이지)

What changed:

- Pipeline 중심이 **CMA-ES black-box 최적화** → **Hypothesis-driven 탐색 + Intent-aware 결정기**로 이동
- 정적 BT 목표 → **달한 루프 학습 시스템** 목표
- v5.0의 4-layer 구조는 유지되지만, 그 위에 **3-stage 시간 축** 이 추가됨 (EXPLORE → LEARN → APPLY → feedback)

What is preserved:

- BFM 이론, Wilson CI, 695 opponent pool, test_suite, TUNABLE_PARAMS auto-discovery
- 기존 코드는 유지되며 역할만 재정의됨

Critical new components:

1. **Hypothesis Miner** (§8) — 데이터에서 자동으로 가설 제안
2. **Counter Selector** (§6.2) — intent → 최적 대응 매핑
3. **Cause Analysis** (§6.4) — 실패를 4-category로 자동 분류 → 피드백
4. **Runtime Feedback** (§7.3) — 매치 데이터가 EXPLORE로 자연 환류

Current priority:

- Sprint A: 측정 인프라 통일 + 태깅
- Sprint B: Hypothesis Miner 최초 2종
- Sprint C-F: 데이터 축적 → Intent model → Counter selector → APPLY

Non-goals (명시):

- CMA-ES를 primary 도구로 사용하지 않음
- Black-box 탐색 의존하지 않음
- Single best BT를 추구하지 않음 (adaptive BT가 목표)